

Úvod do jazyka Python

Čísla, bity, bajty

Vladimír Župka 2021

Použité zdroje informací.....	4
Čísla v počítačích.....	5
Obecný tvar čísla v poziční soustavě.....	5
Číselné soustavy v počítačích.....	5
Bajty (bytes).....	6
Převod z dvojkové do desítkové soustavy.....	6
Převod celých čísel mezi dvojkovou a šestnáctkovou soustavou.....	7
Zjednodušený převod mezi dvojkovou a šestnáctkovou soustavou.....	8
Převody celých čísel z desítkové soustavy.....	8
Příklad převodu do šestnáctkové soustavy.....	8
Příklad převodu do dvojkové soustavy.....	9
Převodní funkce bin() a hex().....	9
Desítková (dekadická, decimální) čísla typu Decimal.....	10
Operace s desítkovými čísly.....	11
Parametry desítkových operací (context).....	12
Převody čísel mezi soustavami.....	13
Převod z dvojkové soustavy do desítkové.....	13
Převod z šestnáctkové soustavy do desítkové.....	14
Převod z desítkové soustavy do dvojkové.....	15
Celá část čísla.....	15
Zlomková část čísla.....	16
Funkce pro převod desítkového čísla na dvojkové vyjádření.....	17
Převod z desítkové soustavy do šestnáctkové.....	18
Celá část čísla.....	18
Zlomková část čísla.....	19
Funkce pro převod desítkového čísla na šestnáctkové vyjádření.....	20
Operace s dvojkovými čísly.....	21
Sčítání dvojkových čísel ručně.....	21
Odčítání dvojkových čísel ručně.....	21
Násobení dvojkových čísel ručně.....	22
Dělení dvojkových čísel ručně.....	23
Dvojková čísla v počítačích.....	24
Dvojkový doplněk.....	24
Rozsahy osmibitových čísel s dvojkovým doplněkem.....	25
Odčítání čísel s dvojkovým doplněkem.....	26
Přetečení výsledku.....	26
Velikosti dvojkových čísel v Pythonu.....	27
Rozsah typu int v 64bitovém počítači.....	27
Rozsah typu float v 64bitovém počítači.....	28
Dvojkové vyjádření.....	28
Desítkové vyjádření.....	28
Rozsah typu float v Pythonu.....	28
Bajtová data.....	29
Bajtový řetězec (bytes).....	29
ASCII znaky a desítková čísla.....	30
ASCII znaky a šestnáctková čísla.....	30
Literál bajtového řetězce.....	31
Kódování UTF-8.....	32
Bajtové pole (bytearray).....	34
Převod čísla typu int na bajtový řetězec a opačně.....	35
Převod čísla typu float na bajtový řetězec a opačně.....	36

Převod float na 8bajtový řetězec.....	36
Převod 8bajtového řetězce na float.....	36
Převod float na 4bajtový řetězec.....	36
Převod 4bajtového řetězce na float.....	36
Převod float na 64 bitů textu.....	37
Převod float na 32 bitů textu.....	37
Převod 64 bitů textu na float.....	37
Převod 32 bitů textu na float.....	37
Bitové operace	38
Bitový součin AND.....	38
Bitový součet OR.....	38
Výlučný bitový součet XOR.....	38
Posuvy vpravo.....	39
Posuvy vlevo.....	39
Zápor NOT.....	39
Testování bitu (AND)	39
Vložení bitu (OR)	39
Binární soubory	40
Soubor celých čísel	40
Vytvoření binárního souboru z celých čísel typu int (C short)	40
Výpočet maxima, součtu a průměru z čísel typu int (C short)	40
Soubor čísel typu float	41
Vytvoření binárního souboru z čísel typu float (C double).....	41
Výpočet maxima z čísel typu float (C double)	41

Použité zdroje informací

Čísla a číselné soustavy: https://web.natur.cuni.cz/~bayertom/images/courses/Prog1/programovani_ciselne_soustavy.pdf

Čísla v počítačích

Obecný tvar čísla v poziční soustavě

Obecně v poziční soustavě o základu z se racionální číslo $r_{(z)}$ vyjádří jako mnohočlen

$$r_{(z)} = a_n \cdot z^n + a_{n-1} \cdot z^{n-1} + \dots + a_1 \cdot z^1 + a_0 \cdot z^0 + a_{-1} \cdot z^{-1} + \dots + a_{-m} \cdot z^{-m}$$

$$\text{nebo } r_{(z)} = \sum_{i=n}^{-m} a_i \cdot z^i,$$

kde

i je celé číslo v rozsahu $n, n-1, \dots, 1, 0, 1, \dots, -m$

a_i je číslice i -tého řádu v soustavě o základu z

z je základ číselné soustavy.

Praktičtější je rozdělit číslo $r_{(z)}$ na celou část $c_{(z)}$ a lomenou část $l_{(z)}$ a každou vyhodnocovat zvlášť.

$$c_{(z)} = a_n \cdot z^n + a_{n-1} \cdot z^{n-1} + \dots + a_1 \cdot z^1 + a_0 \cdot z^0$$

$$l_{(z)} = a_{-1} \cdot z^{-1} + a_{-2} \cdot z^{-2} + \dots + a_{-m+1} \cdot z^{-m+1} + a_{-m} \cdot z^{-m}$$

$$r_{(z)} = c_{(z)} + l_{(z)}, \text{ kde celá část je } c_{(z)} = \sum_{i=n}^0 a_i \cdot z^i \text{ a lomená část je } l_{(z)} = \sum_{i=1}^m a_{-i} \cdot z^{-i}.$$

Číselné soustavy v počítačích

Ve výpočetní technice jsou často používány tyto číselné soustavy:

- desítková (dekadická, decimální)
- dvojková (binární)
- šestnáctková (hexadecimální)
- osmičková (oktalová) – tu pro stručnost vynecháme

Desítková (dekadická, decimální) soustava má 10 číslic: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

Příklad čísla o základu 10:

$$103,05_{(10)} = 1 \cdot 10^2 + 0 \cdot 10^1 + 3 \cdot 10^0 + 0 \cdot 10^{-1} + 5 \cdot 10^{-2}$$

Dvojková (binární) soustava má 2 číslice: 0 1

Příklad čísla o základu 2:

$$101,01_{(2)} = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} = 5 \cdot 25_{(10)}$$

Šestnáctková (hexadecimální) má 16 číslic:

0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

– odpovídající desítkové číslice

Příklad čísla o základu 16:

$$\begin{aligned} a0c,0d_{(16)} &= 10 \cdot 16^2 + 0 \cdot 16^1 + 12 \cdot 16^0 + 0 \cdot 16^{-1} + 13 \cdot 16^{-2} = 10 \cdot 256 + 12 \cdot 1 + 13 \cdot 0,00390625 \\ &= 2572,05078125_{(10)} \end{aligned}$$

Bajty (bytes)

Bajt (byte) je nejmenší adresovací jednotka v soudobých počítačích a také nejmenší jednotka měření objemů dat. Bajt je reprezentován dvojkovým číslem o 8 číslicích, např. $11010110_{(2)}$. Toto dvojkové (binární) číslo lze vyjádřit v ostatních soustavách.

Převod z dvojkové do desítkové soustavy

Stačí vzít jednotlivé číslice zleva (nebo zprava), vynásobit je příslušnou mocninou dvojky a výsledky sečíst.

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
dec	128	64	32	16	8	4	2	1	
	x	x	x	x	x	x	x	x	
bin	1	1	0	1	0	1	1	0	
	↓	↓	↓	↓	↓	↓	↓	↓	
dec	128	64	0	16	0	4	2	0	$\Sigma = 214_{(10)}$

Převod celých čísel mezi dvojkovou a šestnáctkovou soustavou

Převod stejného čísla $11010110_{(2)}$ z dvojkové do šestnáctkové soustavy je podobný, jen je potřeba umět šestnáctková čísla sčítat – podle šestnáctkové součtové tabulky:

+	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	10
2	3	4	5	6	7	8	9	a	b	c	d	e	f	10	11
3	4	5	6	7	8	9	a	b	c	d	e	f	10	11	12
4	5	6	7	8	9	a	b	c	d	e	f	10	11	12	13
5	6	7	8	9	a	b	c	d	e	f	10	11	12	13	14
6	7	8	9	a	b	c	d	e	f	10	11	12	13	14	15
7	8	9	a	b	c	d	e	f	10	11	12	13	14	15	16
8	9	a	b	c	d	e	f	10	11	12	13	14	15	16	17
9	a	b	c	d	e	f	10	11	12	13	14	15	16	17	18
a	b	c	d	e	f	10	11	12	13	14	15	16	17	18	19
b	c	d	e	f	10	11	12	13	14	15	16	17	18	19	1a
c	d	e	f	10	11	12	13	14	15	16	17	18	19	1a	1b
d	e	f	10	11	12	13	14	15	16	17	18	19	1a	1b	1c
e	f	10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d
f	10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d	1e

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
hex	80	40	20	10	8	4	2	1	
	x	x	x	x	x	x	x	x	
bin	1	1	0	1	0	1	1	0	
	↓	↓	↓	↓	↓	↓	↓	↓	
hex	80	40	0	10	0	4	2	0	$\Sigma = d6$

Při sčítání se postupuje stejně jako v desítkové soustavě, přenos jedničky o řád se provede, když součet číslic překročí nejvyšší číslici f. Např. $bca + de = ca8$:

$$\begin{array}{r}
 \text{b c a} \\
 + \text{ d e} \\
 \hline
 \text{1 8} \\
 \text{1 9} \\
 \hline
 \text{b} \\
 = \text{c a 8}
 \end{array}$$

Zjednodušený převod mezi dvojkovou a šestnáctkovou soustavou

Převod mezi dvojkovou a šestnáctkovou soustavou je ještě zjednodušen tím, že $16 = 2^4$. To znamená, že *čtveřice* dvojkových číslic tvoří jednu šestnáctkovou číslici:

0001 = 1	1010 = a
0010 = 2	1011 = b
0011 = 3	1100 = c
0100 = 4	1101 = d
0101 = 5	1110 = e
0110 = 6	1111 = f
0111 = 7	
1000 = 8	
1000 = 9	

Největší číslo v bajtu je $11111111_{(2)}$ nebo $ff_{(16)}$, čili $255_{(10)}$. V Pythonu se základ soustavy vyjadřuje předponou `0b` u binární, `0x` u šestnáctkové (hexadecimální):

$$255 = 0b11111111 = 0xff$$

Převody celých čísel z desítkové soustavy

Převody z dvojkové a šestnáctkové soustavy do desítkové jsou celkem přímočaré. Opačné převody jsou trochu náročnější, vyžadují operace postupného dělení se zbytkem.

Příklad převodu do šestnáctkové soustavy

Převod čísla 572 do šestnáctkové soustavy postupným dělením a odečítáním zbytku.

$572 : 16 = 30$	$35 : 16 = 2$	$2 : 16 = 0$
$\underline{-480}$	$\underline{-32}$	zbytek 2 = 2
$92 : 16 = 5$	$3 : 16 = 0$	
$\underline{-80}$	zbytek 3 = 3	
$12 : 16 = 0$		
zbytek 12 = c		

Výsledné šestnáctkové číslice seřadíme *obráceně*: **23c**. Je tedy $572_{(10)} = 23c_{(16)}$.

Příklad převodu do dvojkové soustavy

Převod čísla 14 do dvojkové soustavy postupným dělením a odečítáním zbytku.

$$\begin{array}{rcll} 141 & \% & 2 & = 1 \\ \hline -1 & & & \\ 140 & : & 2 & = 70 \% 2 = 0 \\ \hline -0 & & & \\ 70 & : & 2 & = 35 \% 2 = 1 \\ \hline -1 & & & \\ 34 & : & 2 & = 17 \% 2 = 1 \\ \hline -1 & & & \\ 16 & : & 2 & = 8 \% 2 = 0 \\ \hline -0 & & & \\ 8 & : & 2 & = 4 \% 2 = 0 \\ \hline -0 & & & \\ 4 & : & 2 & = 2 \% 2 = 0 \\ \hline -0 & & & \\ 2 & : & 2 & = 1 \% 2 = 1 \\ \hline -1 & & & \\ 0 & & & \end{array}$$

Výsledné dvojkové číslice seřadíme *obráceně*: **10001101**. Je tedy $141_{(10)} = 10001101_{(2)}$.

Převodní funkce `bin()` a `hex()`

Tyto standardní funkce usnadňují převody *celých čísel* mezi soustavami.

Převod celého desítkového čísla do dvojkové a šestnáctkové soustavy.

```
print(bin(3287))      # 0b110011010111
print(hex(3287))      # 0xcd7
```

Převod celého dvojkového čísla do šestnáctkové a desítkové soustavy.

```
print(hex(0b11111111)) # 0xff
print(0b11111111)      # 255
```

Převod celého šestnáctkového čísla do dvojkové a desítkové soustavy.

```
print(bin(0xff))      # 0b11111111
print(0xff)           # 255
```

Argumentem může být libovolně velké celé číslo, i záporné. Zápisy 3287, 0b11111111 a 0xff jsou *literály* celých čísel.

Desítková (dekadická, decimální) čísla typu `Decimal`

Výpočty s typem `float` mají některé problematické rysy. Číslo je uloženo jako *binární* v paměťové oblasti o 64 bitech, zatímco uživatel je potřebuje vidět v *desítkové* podobě. Potíže spočívají v nepřesnosti převodu do desítkové soustavy a v nedostatečném počtu platných číslic zejména u komerčních aplikací.

```
print(1.1 + 1.3) # 2.4000000000000004
```

Vlastnosti čísel s pohyblivou řádovou čárkou (tečkou) jsou podrobně rozebrány na stránce <https://docs.python.org/3/tutorial/floatpoint.html#tut-fp-issues>.

Desítková aritmetika v Pythonu tyto potíže odstraňuje. Není tak rychlá jako dvojková aritmetika, ale zato jsou její výsledky přesné a rozsah čísel je libovolně velký. Je realizovaná v modulu **`decimal`** třídou **`Decimal`**: <https://docs.python.org/3/library/decimal.html>.

```
from decimal import *
```

Desítkové číslo se vytvoří voláním *konstruktoru třídy* `Decimal` s argumentem typu `str`, `int` nebo `float`. (O třídách a konstruktorech pojednává další část kurzu.) Desítkové číslo může tedy být jak celé (integer), tak zlomkové (floating point).

```
Decimal() # 0
Decimal('0') # 0
Decimal(1) # 1
Decimal('1') # 1
Decimal('3.14159') # 3.14159
```

Pro zkrácení příkazů si můžeme definovat alternativní jméno *třídy*, např. `D`, které můžeme používat střídavě se jménem `Decimal`.

```
D = Decimal # definice alternativního jména třídy
```

Argument typu `float` může dávat odlišnou hodnotu desítkového čísla.

```
D(1) # 1
D(1.75) # 1.75
D(3.14159) # 3.1415899999999998261834005243144929409027099609375
```

Poslední výsledek vyjadřuje co nejpřesněji číslo `3.14159` typu `float` (na 51 platných číslic). Kontrast stejných čísel typu `float` a `Decimal` je vidět z následujícího příkladu.

```
print(1.1 + 1.3) # 2.4000000000000004
print(Decimal(1.1 + 1.3)) # 2.4000000000000003552713678800500929355621337890625
print(Decimal('1.1') + Decimal('1.3')) # 2.4
```

Operace s desítkovými čísly

Desítková čísla typu Decimal vyhovují téměř všem operacím. Uvádíme jen několik příkladů.

```
decimals = [Decimal(x) for x in ('20.34', '4.36', '3.23', '1.01')]
print(decimals)
[Decimal('20.34'), Decimal('4.36'), Decimal('3.23'), Decimal('1.01')]
```

```
a, b, c = decimals[:3] # rozbalení (unpacking) prvních členů seznamu
```

Převod desítkového čísla do jiných typů

```
print(repr(str(a))) # '20.34'
print(repr(float(a))) # 20.34
print(repr(int(a))) # 20
```

Zaokrouhlení

```
print(repr(round(a, 1))) # Decimal('20.3') – zaokrouhlení na 1 des. místo
```

Násobení a dělení

```
print(repr(a * 2)) # Decimal('40.68')
print(repr(a * b)) # Decimal('88.6824')
print(repr(a / 2)) # Decimal('10.17')
print(repr(a % 2)) # Decimal('0.34')
```

Standardní funkce

```
print(min(decimals)) # 1.01 – minimum
print(sorted(decimals)) # třídění vzestupně do nového seznamu
[Decimal('1.01'), Decimal('3.23'), Decimal('4.36'), Decimal('20.34')]
```

Mocniny a logaritmy

```
print(D('10')**2) # 100 = 102
print(D('100').log10()) # 2 = log10(100)
print(D('10').ln()) # 2.302585092994045684017991455 = loge(10)
x = D('2.302585092994045684017991455')
print(x.exp()) # 10.000000000000000000000000000000 = ex
```

Národní prostředí

```
import locale
locale.setlocale(locale.LC_ALL, 'cs_CZ')
print(locale.currency(Decimal('-356789.14'), grouping=True)) # -356 789,14 Kč
```

Parametry desítkových operací (context)

Výpočetní parametry pro desítkové operace se zjistí funkcí `getContext()`.

```
cont = getcontext()  
print(cont)
```

```
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999,
capitals=1, clamp=0, flags=[Inexact, FloatOperation, Rounded],
traps=[InvalidOperation, DivisionByZero, Overflow])
```

Nejdůležitější parametry jsou

prec	přesnost – maximální počet platných číslic, standardně 28
rounding	standardně zaokrouhlení poloviny na nejbližší <i>sudou</i> číslici, další v dokumentaci
	<code>print(round(D('2.345'), 2))</code> # 2.34
Emin, Emax	minimální a maximální exponent

Parametry můžeme změnit funkcí `setcontext()`, např. přesnost z 28 na 45 platných číslic.

```
cont.prec = 45
setcontext(cont)
print(cont)
```

```
Context(prec=45, rounding=ROUND_HALF_EVEN, ...)
```

Nejmenší kladné desítkové číslo s pohyblivou čárkou je 2^{-1022} neboli $\frac{1}{2^{1022}}$. Zde na 45 platných číslic. Viz [dále](#).

```
print(D('2')** -1022)
```

2.22507385850720138309023271733240406421921598E-308

Celé kladné desítkové číslo na 45 platných číslic: na začátku 1, pak nuly a na konci 9

```
print(D('10')*10**43 + D('9'))
```

10009

Převod z šestnáctkové soustavy do desítkové

$$c_{(10)} = h_n \cdot 16^n + h_{n-1} \cdot 16^{n-1} + \dots + h_1 \cdot 16^1 + h_0 \cdot 16^0$$

$$l_{(10)} = h_{-1} \cdot 16^{-1} + h_{-2} \cdot 16^{-2} + \dots + h_{-m+1} \cdot 16^{-m+1} + h_{-m} \cdot 16^{-m}$$

$$r_{(10)} = c_{(10)} + l_{(10)}, \text{ kde celá část je } c_{(10)} = \sum_{i=0}^n h_i \cdot 16^i \text{ a lomená část je } l_{(10)} = \sum_{i=1}^m h_{-i} \cdot 16^{-i}.$$

```
def h_d(h):
    """ Funkce pro převod šestnáctkové číslice na desítkové číslo """
    chr = {'0': 0, '1': 1, '2': 2, '3': 3, '4': 4, '5': 5,
           '6': 6, '7': 7, '8': 8, '9': 9, 'a': 10, 'b': 11,
           'c': 12, 'd': 13, 'e': 14, 'f': 15, '.': '.'}
    return chr[h]
```

```
def hex_to_dec(hex):
    """ Funkce pro převod šestnáctkového čísla na desítkový tvar """
    assert type(hex) == str, "Argument musí být řetězec (typ str)."
    assert hex.count('.') < 2, "Tečka může být jen jedna."

    for h in hex: # kontrola znaků v hexadecimálním řetězci
        try:
            h_d(h)
        except KeyError:
            print(f'Chybný znak "{h}" v argumentu {hex}.')
            return '' # aby se netisklo None

    index = hex.find('.') # hledám index tečky
    if index > -1: # když se tečka našla
        celky = hex[:index] # celky před tečkou
        zlomek = hex[index+1:] # zlomek za tečkou
    else: # když se tečka nenašla, mám jen celky
        celky = hex # celky jsou celý parametr a zlomek je prázdný
        zlomek = ''

    n = len(celky)
    m = len(zlomek)
    dec = Decimal('0')
    sestnact = Decimal('16')
    for i in range(0, n): # přičítám mocniny 16 od vyšších k nulté
        dec += Decimal(h_d(celky[n-1-i])) * sestnact ** i
    for i in range(0, m): # přičítám mocniny 1/16 od začátku zlomku
        dec += Decimal(h_d(zlomek[i])) * sestnact ** (-i-1)
    return dec
```

```
print(hex_to_dec('82.ef15')) # 130.9339141845703125
print(hex_to_dec('a0c.0d')) # 2572.05078125
print(hex_to_dec('a0č.0d')) # Chybný znak "č" v argumentu a0č.0d. Končím.
```

Převod z desítkové soustavy do dvojkové

Celá část čísla

Celou část čísla dělíme dvěma a zjistíme zbytek (operace %). Zbytek (0 nebo 1) zapíšeme a odečteme od čísla (dělení). Rozdíl znovu dělíme dvěma a zjistíme zbytek atd. a opakujeme až po poslední zbytek.

$$2572 \% 2 = 0$$

$$\begin{array}{r} \underline{-0} \\ 2572 : 2 = 1286 \end{array} \% 2 = 0$$

$$\begin{array}{r} \underline{-0} \\ 1286 : 2 = 643 \end{array} \% 2 = 1$$

$$\begin{array}{r} \underline{-1} \\ 642 : 2 = 321 \end{array} \% 2 = 1$$

$$\begin{array}{r} \underline{-1} \\ 320 : 2 = 160 \end{array} \% 2 = 0$$

$$\begin{array}{r} \underline{-0} \\ 160 : 2 = 80 \end{array} \% 2 = 0$$

$$80$$

$$\begin{array}{r} \underline{-0} \\ 80 : 2 = 40 \end{array} \% 2 = 0$$

$$\begin{array}{r} \underline{-0} \\ 40 : 2 = 20 \end{array} \% 2 = 0$$

$$\begin{array}{r} \underline{-0} \\ 20 : 2 = 10 \end{array} \% 2 = 0$$

$$\begin{array}{r} \underline{-0} \\ 10 : 2 = 5 \end{array} \% 2 = 1$$

$$\begin{array}{r} \underline{-1} \\ 4 : 2 = 2 \end{array} \% 2 = 0$$

$$\begin{array}{r} \underline{-0} \\ 2 : 2 = 1 \end{array} \% 2 = 1$$

$$\begin{array}{r} \underline{-1} \\ 0 \end{array}$$

Výsledné dvojkové číslice seřadíme *od poslední po první*: 101000001100.

Je tedy $2572_{(10)} = 101000001100_{(2)}$.

Zlomková část čísla

Zlomkovou část také postupně dělíme, ale číslem $2^{-1} = \frac{1}{2} = 0,5$, čili *násobíme dvěma*. Zjišťujeme dvojnásobky a zbytky. Z dvojnásobku odstraníme celou část (0 nebo 1), kterou zapíšeme. Zbylou zlomkovou část (zbytek) znovu násobíme dvěma atd. Končíme buď tehdy, když je zbytek nulový nebo po stanoveném maximálním počtu kroků. Převedené zlomkové číslo totiž nemusí mít konečný počet polovinových míst. Např. desítkové číslo $0,2$ je dvojkové číslo $0,001100110011\dots$, kde se opakuje skupina 0011.

Následující příklad ukazuje konečný postup dělení, když zbytek vyjde nulový.

,05078125	x 2	=	0,1015625	0
,1015625	x 2	=	0,203125	0
,203125	x 2	=	0,40625	0
,40625	x 2	=	0,8125	0
,8125	x 2	=	1,625	1
,625	x 2	=	1,25	1
,25	x 2	=	0,5	0
,5	x 2	=	1	1
zbytek = 0				$0,05078125 = 0,00001101$

Další příklad ukazuje nekonečný postup, když se opakuje skupina zbytků a tedy skupina výsledných číslic.

,7	x 2	=	1,4	1
,4	x 2	=	0,8	0
,8	x 2	=	1,6	1
,6	x 2	=	1,2	1
,2	x 2	=	0,4	0
,4	x 2	=	0,8	0
...				
zbytek 4 znovu				$0,7 = 0,1011001100110\dots$

Funkce pro převod desítkového čísla na dvojkové vyjádření

```
from decimal import *
def dec_to_bin(cislo):
    """Funkce pro převod desítkového čísla na dvojkový tvar"""
    assert type(cislo) is Decimal, "Argument musí být číslo typu decimal."
    assert cislo >= 0, "Číslo nesmí být záporné."
    nula = Decimal()
    jedna = Decimal('1')
    dva = Decimal('2')
    cislice = [] # seznam dvojkových číslic znakově vyjádřených
    celky = cislo // jedna # celá část čísla
    zlomek = cislo % jedna # zlomková část čísla

    # celá část čísla
    while True: # získám dvojkové číslice v obráceném pořadí řádů
        if celky >= jedna:
            zbytek = celky % dva # zbytek po dělení dvěma 1 nebo 0
            cislice.append(str(zbytek)) # zbytek do seznamu
            celky = (celky - zbytek) / dva # odečtu zbytek a dělím 2
        else:
            break # končím když celá část je 0
    cislice.reverse() # obrátím pořadí v seznamu číslic

    # zlomková část čísla
    if zlomek != 0:
        cislice.append('.') # přidám do seznamu desetinnou tečku
        for i in range(200): # maximální počet číslic zlomku
            zlomek = zlomek * dva # dvojnásobek zlomku
            cislice.append(str(zlomek // jedna)) # zapíšu celou část zlomku
            zbytek = zlomek % jedna # zbytek po dělení jednou
            if zbytek == nula:
                break # končím, když je zbytek nulový
            else:
                zlomek = zbytek # zbytek se stane zlomkem
    # sestavím dvojkové číslo
    bin = ''
    for i in cislice:
        bin += i
    return bin
```

```
cislo = Decimal('2572.05078125')
```

```
bin = dec_to_bin(cislo)
print(f'Desítkové číslo {cislo} = dvojkové číslo {bin}')
```

```
Desítkové číslo 2572.05078125 = dvojkové číslo 101000001100.00001101
```

Převod z desítkové soustavy do šestnáctkové

Celá část čísla

Celou část čísla dělíme postupně šestnácti a zjistíme podíl a zbytek. Zbytek zapíšeme a převedeme na šestnáctkovou číslici. Z podílu oddělíme celou část, dělíme ji šestnácti a zjistíme podíl a zbytek atd. až po poslední zbytek.

$$2572 : 16 = 100$$

$$\underline{-1600}$$

$$972 : 16 = 60$$

$$\underline{-960}$$

$$12 : 16 = 0$$

celá část 160

zbytek 12 = **c**

$$160 : 16 = 10$$

$$\underline{-160}$$

$$0 : 16 = 0$$

celá část 10

zbytek 0 = **0**

$$10 : 16 = 0$$

celá část 0

zbytek 10 = **a**

Postup pomocí kalkulačky:

$$2572 : 16 = 160,75 = 160, \text{ zbytek} = 0,75 \times 16 = 12 = \mathbf{c}$$

$$160 : 16 = 10, \text{ zbytek } \mathbf{0}$$

$$10 : 16 = 0, \text{ zbytek } 10 = \mathbf{a}$$

Výsledné šestnáctkové číslice seřadíme *od poslední po první*: **a0c**.

Je tedy $2572_{(10)} = \mathbf{a0c}_{(16)}$.

Zlomková část čísla

Zlomkovou část také postupně dělíme, ale číslem $16^{-1} = 1/16 = 0,0625$, čili *násobíme* šestnácti. Zjišťujeme násobky a zbytky. Z násobku odstraníme celou část, kterou zapíšeme a převedeme na šestnáctkovou číslici. Zbylou zlomkovou část (zbytek) znovu násobíme šestnácti atd. Končíme buď tehdy, když je zbytek nulový nebo po stanoveném maximálním počtu kroků. Převedené zlomkové číslo totiž nemusí mít konečný počet šestnáctinových míst. Např. desítkové číslo $0,2$ je šestnáctkové číslo $0,333\dots$ s nekonečným počtem trojek.

Následující příklad ukazuje konečný postup dělení, když zbytek vyjde nulový.

$$\begin{array}{lcl} \textbf{,05078125} \times 16 & = & \textbf{0,8125} \quad \textbf{0} \\ \textbf{,8125} \times 16 & = & \textbf{13,0} \quad \textbf{d} \\ \text{zbytek} & = & 0 \end{array} \qquad 0,05078125 = 0,0d$$

Další příklad ukazuje nekonečný postup, když se opakuje skupina zbytků a tedy skupina výsledných číslic (c28f5).

$$\begin{array}{lcl} \textbf{,11} \times 16 & = & \textbf{1,76} \quad \textbf{1} \\ \textbf{,76} \times 16 & = & \textbf{12,16} \quad \textbf{c} \\ \textbf{,16} \times 16 & = & \textbf{2,56} \quad \textbf{2} \\ \textbf{,56} \times 16 & = & \textbf{8,96} \quad \textbf{8} \\ \textbf{,96} \times 16 & = & \textbf{15,36} \quad \textbf{f} \\ \textbf{,36} \times 16 & = & \textbf{5,76} \quad \textbf{5} \\ \textbf{,76} \times 16 & = & \textbf{12,16} \quad \textbf{c} \\ \textbf{,16} \times 16 & = & \textbf{2,56} \quad \textbf{2} \\ \dots & & \\ \text{zbytek } \textbf{16} & \text{znovu} & \end{array} \qquad 0,11 = 0,1\textbf{c28f5c28f5}\dots$$

Funkce pro převod desítkového čísla na šestnáctkové vyjádření

```
from decimal import *
```

```
def d_h(dec):  
    """ Funkce pro převod desítkového čísla na šestnáctkovou číslici """  
    idx = int(dec) # převod decimal na int  
    return '0123456789abcdef'[idx] # vyhledání hex číslice podle indexu
```

```
def dec_to_hex(cislo):  
    """Funkce pro převod desítkového čísla na šestnáctkový tvar"""  
    assert type(cislo) is Decimal, "Argument musí být číslo typu decimal."  
    assert cislo >= 0, "Číslo nesmí být záporné."  
    nula = Decimal()  
    jedna = Decimal('1')  
    sestnact = Decimal('16')  
    cislice = [] # seznam šestnáctkových číslic  
    celky = cislo // jedna # celá část čísla  
    zlomek = cislo % jedna # zlomková část čísla  
  
    # celá část čísla  
    while True: # získám šestnáctkové číslice v obráceném pořadí řádů  
        if celky >= jedna:  
            zbytek = celky % sestnact # zbytek po dělení šestnáctí 0 až 15  
            cislice.append(d_h(zbytek)) # zbytek do seznamu  
            celky = celky // sestnact # vypočtu celou část podílu  
        else:  
            break # končím když celá část je 0  
    cislice.reverse() # obrátím pořadí v seznamu číslic  
  
    # zlomková část čísla  
    if zlomek != 0:  
        cislice.append('.') # přidám do seznamu desetinnou tečku  
        for i in range(100):  
            zlomek = zlomek * sestnact # šestnáctinásobek zlomku  
            cislice.append(d_h(zlomek // jedna)) # zapíšu celou část zlomku  
            zbytek = zlomek % jedna # zbytek po dělení jednou  
            if zbytek == nula:  
                break # končím, když je zbytek nulový  
            else:  
                zlomek = zbytek # zbytek se stane zlomkem  
    # sestavím šestnáctkové číslo  
    hex = ''  
    for i in cislice:  
        hex += i  
    return hex
```

```
cislo = Decimal('+2572.05078125') # a0c.0d
```

```
hex = dec_to_hex(cislo)  
print(f'Desítkové číslo {cislo} = šestnáctkové číslo {hex}')
```

```
Desítkové číslo 2572.05078125 = šestnáctkové číslo a0c.0d
```


Vychází-li *rozdílný*, provedeme *dvojkový přenos*, tj. připojíme jedničku 1 ke *spodní* číslici o dvojkový řád výš, tedy o jeden sloupec vlevo. K *horní* číslici přičteme dvojku (vyznačenou žlutě 10 nad horní číslicí). Od tohoto součtu, který vychází jako 10 nebo 11, odečteme *spodní* číslici a *případnou* přidanou jedničku. Výsledek, který může být 0 nebo 1, zapíšeme červeně, a přejdeme na další sloupec doleva.

Příklady: Větší minus menší.

$$\begin{array}{r}
 14_{(10)} \quad \quad \quad \begin{array}{c} 10 \quad 10 \\ 1 \quad 1 \quad 1 \quad 0 \end{array} \\
 -11_{(10)} - \underline{1 \quad 0 \quad 1 \quad 1} \\
 \quad \quad \quad \begin{array}{c} 1 \quad 1 \\ 1 \quad 1 \\ 0 \end{array} \quad \begin{array}{c} 10-1=1, \text{ přenos} \\ 11-1-1=1, \text{ přenos} \\ 1-0-1=0 \end{array} \\
 \quad \quad \quad \begin{array}{c} 0 \\ 0 \end{array} \quad \begin{array}{c} 1-1=0 \\ 0 \quad 0 \quad 1 \quad 1 \end{array} = 3_{(10)}
 \end{array}$$

$$\begin{array}{r}
 14_{(10)} \quad \quad \quad \begin{array}{c} 10 \\ 1 \quad 1 \quad 1 \quad 0 \end{array} \\
 -9_{(10)} - \underline{1 \quad 0 \quad 0 \quad 1} \\
 \quad \quad \quad \begin{array}{c} 1 \quad 1 \\ 0 \\ 1 \end{array} \quad \begin{array}{c} 10-1=1, \text{ přenos} \\ 1-0-1=0 \\ 1-0=1 \end{array} \\
 \quad \quad \quad \begin{array}{c} 0 \\ 0 \end{array} \quad \begin{array}{c} 1-1=0 \\ 0 \quad 1 \quad 0 \quad 1 \end{array} = 5_{(10)}
 \end{array}$$

$$\begin{array}{r}
 20_{(10)} \quad \quad \quad \begin{array}{c} 10 \quad 10 \quad 10 \\ 1 \quad 0 \quad 1 \quad 0 \quad 0 \end{array} \\
 -11_{(10)} - \underline{0 \quad 1 \quad 0 \quad 1 \quad 1} \\
 \quad \quad \quad \begin{array}{c} 1 \quad 1 \\ 1 \quad 0 \\ 0 \end{array} \quad \begin{array}{c} 10-1=1, \text{ přenos} \\ 10-1-1=0, \text{ přenos} \\ 1-0-1=0 \end{array} \\
 \quad \quad \quad \begin{array}{c} 1 \quad 1 \\ 0 \end{array} \quad \begin{array}{c} 10-1=1, \text{ přenos} \\ 1-0-1=0 \end{array} \\
 \quad \quad \quad \begin{array}{c} 0 \\ 0 \end{array} \quad \begin{array}{c} 1-0-1=0 \\ 0 \quad 1 \quad 0 \quad 0 \quad 1 \end{array} = 9_{(10)}
 \end{array}$$

Příklady: Menší minus větší. Rozdíl se vypočte jako obrácený rozdíl s opačným znaménkem:
 $m - v = -(v - m)$.

$$\begin{array}{r}
 m = +10_{(10)} \quad + \quad 1 \quad 0 \quad 1 \quad 0 \\
 v = -12_{(10)} \quad - \quad \underline{1 \quad 1 \quad 0 \quad 0} \\
 \quad \quad \quad - \quad 0 \quad 0 \quad 1 \quad 0 \quad = -2_{(10)}
 \end{array}$$

$$\begin{array}{r}
 m = +11_{(10)} \quad + \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \\
 v = -20_{(10)} \quad - \quad \underline{1 \quad 0 \quad 1 \quad 0 \quad 0} \\
 \quad \quad \quad - \quad 0 \quad 1 \quad 0 \quad 0 \quad 1 \quad = -9_{(10)}
 \end{array}$$

Násobení dvojkových čísel ručně

Násobíme první operand (násobenec) postupně číslicemi druhého operandu (násobitele) zprava doleva a výsledek vždy zarovnáme vpravo podle řádu číslice druhého operandu.

$$\begin{array}{r}
 21_{(10)} \quad \quad \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \\
 4_{(10)} \quad \times \quad \underline{1 \quad 0 \quad 0} \\
 \quad \quad \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad \times 0 \\
 \quad \quad \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad \times 0 \\
 \quad \quad \underline{1 \quad 0 \quad 1 \quad 0 \quad 1} \quad \times 1 \\
 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 = 84_{(10)} = 64 + 16 + 4
 \end{array}$$

Dělení dvojkových čísel ručně

Od dělence postupně odečítáme dělitele a výsledek 1 nebo 0 zapíšeme vedle dělence. Dělitele pak posuneme o číslici doprava a odečteme atd. až zbyde nula. Když poslední číslice dělitele poprvé překročí hranici dělence, zapíšeme řádovou čárku pro oddělení zlomku. K dělenci připíšeme nulu, když posuneme dělitele doprava ve zlomkové části.

$$\begin{array}{r}
 21_{(10)} : 4_{(10)} \quad 1 \ 0 \ 1 \ 0 \ 1 : 1 \ 0 \ 0 = 101,01 = 5,25_{(10)} \\
 \underline{- 1 \ 0 \ 0} \qquad \qquad 1 \\
 0 \ 0 \ 1 \ 0 \ 1 \\
 \underline{- 1 \ 0 \ 0} \qquad \qquad 0 \\
 0 \ 0 \ 1 \ 0 \ 1 \\
 \underline{- 1 \ 0 \ 0} \qquad \qquad 1 \\
 0 \ 0 \ 0 \ 0 \ 1 \ 0 \\
 \underline{- 1 \ 0 \ 0} \qquad \qquad , \ 0 \\
 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \\
 \underline{- 1 \ 0 \ 0} \qquad \qquad 1 \\
 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0
 \end{array}$$

$$\begin{array}{r}
 2_{(10)} : 3_{(10)} \quad 1 \ 0 : 1 \ 1 = 0,101010\dots \text{ opakuje se skupina } 01 \\
 \underline{- 1 \ 1} \qquad \qquad 0 \\
 1 \ 0 \ 0 \qquad \qquad , \\
 \underline{- 1 \ 1} \qquad \qquad 1 \\
 0 \ 0 \ 1 \ 0 \\
 \underline{- 1 \ 1} \qquad \qquad 0 \\
 0 \ 0 \ 1 \ 0 \ 0 \\
 \underline{- 1 \ 1} \qquad \qquad 1 \\
 0 \ 0 \ 0 \ 0 \ 1 \ 0 \\
 \underline{- 1 \ 1} \qquad \qquad 0 \\
 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \\
 \underline{- 1 \ 1} \qquad \qquad 1 \\
 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \\
 \underline{- 1 \ 1} \qquad \qquad 0 \\
 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \\
 \dots \text{ atd.}
 \end{array}$$

$$0,101010\dots = 0,5 + 0,125 + 0,03125 + 0,0078125 + \dots = 0,6640625 + \dots = 0,6\bar{6}_{(10)}$$

Když se skupiny výsledných číslic opakují, je výsledkem nekonečná zlomková část. Převod do desítkové soustavy je nepřesný. Přesný by byl jen za absurdního předpokladu, že výsledek můžeme spočítat na nekonečný počet desetinných míst.

Dvojková čísla v počítačích

Dvojková čísla v počítači někdy obsahují znaménko, někdy ne. Číslo bez znaménka v jednom bajtu může být 0 až 255, nebo 2^0 až 2^8-1 , nebo 0000 0000 až 1111 1111. Považuje se za kladné. Běžně se však pro aritmetické operace ve výrazech nepoužívá.

Pro aritmetické operace je nutné rozeznat, zda je číslo kladné nebo záporné. Znaménko je obvykle určeno prvním bitem (nejvyššího řádu). 0 znamená kladné číslo, 1 znamená záporné číslo.

Jeden způsob uložení čísel je takový, že největší kladné číslo v jednom bajtu je 0111 1111 = +127 a nejmenší záporné číslo je 1111 1111 = -127. Nulové číslo tak může být buď kladné nebo záporné. Tento způsob se používá v Pythonu.

Dvojkový doplněk

Druhý způsob uložení definuje kladné číslo rovněž tak, že nejvyšší bit je 0, ale **záporné číslo** vzniká jako **dvojkový doplněk** kladného čísla. Dvojkový doplněk kladného čísla vznikne tak, že jedničky nahradíme nulami a nuly jedničkami (jedničkový doplněk), a pak k výsledku přičteme jedničku na posledním místě. (Jedničkový doplněk se také nazývá bitová negace.)

1	0000 0001	
	1111 1110	jedničkový doplněk čísla 1
	<u> +1</u>	přičtení jedničky
-1	1111 1111	dvojkový doplněk čísla 1, záporné číslo -1

Dvojkový doplněk dostaneme také tak, když od *jedničky* na místě osmé mocniny dvou ($2^8 = 256$) odečteme původní číslo.

	2^8	
256	1 0000 0000	jednička na vyšším řádu
	<u> -1</u>	odečtení čísla
-1	1111 1111	dvojkový doplněk čísla

Ze **záporného** čísla vznikne **kladné** číslo obráceným postupem. Od záporného čísla (má jedničku na prvním místě) odečteme jedničku (na posledním místě) a provedeme jedničkový doplněk (nahradíme jedničky nulami a nuly jedničkami).

-1	1111 1111	záporné číslo -1 (dvojkový doplněk čísla 1)
	<u> -1</u>	odečtení jedničky
	1111 1110	
1	0000 0001	jedničkový doplněk, výsledek je číslo 1

Rozsahy osmibitových čísel s dvojkovým doplňkem

První bit nulový a ostatní jedničkové představují největší kladné číslo, první bit jedničkový a ostatní nulové představují nejmenší záporné číslo. Maximální rozsah číselných hodnot v jednom bajtu je tedy -128 až $+127$ neboli -2^8 až 2^8-1 .

$-128 = 1000\ 0000$ minimum

$+127 = 0111\ 1111$ maximum

Analogicky, celá čísla obvyklých velikostí mají rozsahy:

2 bajty, 16 bitů $-32\ 768 \dots +32\ 767$

4 bajty, 32 bitů $-2\ 147\ 483\ 648 \dots +2\ 147\ 483\ 647$

8 bajtů, 64 bitů $-9\ 223\ 372\ 036\ 854\ 775\ 808 \dots +9\ 223\ 372\ 036\ 854\ 775\ 807$

Odčítání čísel s dvojkovým doplňkem

Odčítání čísel se provádí tak, že k prvnímu se *přičte dvojkový doplněk* druhého.

Příklad: 74 - 25 neboli 0100 1010 - 0001 1001

$$\begin{array}{r} 74 \quad 0100 \ 1010 \\ + \quad \underline{1110 \ 0111} \quad \text{dvojkový doplněk 25} \\ \hline 1 \ 0011 \ 0001 = 49 \end{array}$$

Jednička, která vyšla mimo rozsah osmi bitů, představuje tzv. *přetečení* výsledku a zahazuje se, když znaménko vyšlo kladné (0).

Přetečení výsledku

Při sčítání kladných čísel může výsledek vyjít záporný.

$$127 + 1 = ?$$

$$\begin{array}{r} 127 \quad 0111 \ 1111 \\ +1 \quad \underline{0000 \ 0001} \\ \hline 1 \ 0000 \ 0000 = -128 \end{array}$$

$$127 + 127 = ?$$

$$\begin{array}{r} 127 \quad 0111 \ 1111 \\ +127 \quad \underline{0111 \ 1111} \\ \hline 1 \ 1111 \ 1110 = -2 \end{array}$$

Při sčítání záporných čísel dochází k přetečení výsledku, když se přenos jedničky dostane mimo rozsah čísla.

$$-128 - 1 = ?$$

$$\begin{array}{r} -128 \quad 1000 \ 0000 \\ +(-1) \quad \underline{1111 \ 1111} \\ \hline 1 \ 0111 \ 1111 = 127 \end{array}$$

$$-128 - 128 = ?$$

$$\begin{array}{r} -128 \quad 1000 \ 0000 \\ +(-128) \quad \underline{1000 \ 0000} \\ \hline 1 \ 0000 \ 0000 = 256 \end{array}$$

S přetečením se v různých programovacích jazycích zachází různě. Například jazyk C se o přetečení vůbec nestará a může dojít k přepsání paměti mimo číslo. Jazyk Python přetečení vůbec nezná. Výsledek operace je tak velký, jak je potřeba. Výše uvedené příklady pak dávají výsledky

$$+127 + 1 = +128 \quad +127 + 127 = +254$$

$$-128 - 1 = -129 \quad -128 - 128 = -256$$

Poznámka: Příklady jsou zkrácené na 8 bitů kvůli přehlednosti. Ve skutečnosti dnes žádný počítačový hardware nepracuje s osmibitovými čísly, ale s 64bitovými čísly.

Rozsah typu `float` v 64bitovém počítači

Viz odkaz *IEEE 754 double-precision binary floating-point format: binary64*.

Dvojkové vyjádření

[illegible]

Červená jednička s tečkou není součástí bitové reprezentace čísla, představuje celé číslo 1, se kterým se počítá při převodech do desítkového tvaru.

Exponent je vyjádřen v tzv. biased (zkresleném) tvaru, a to takovém, že ke skutečnému exponentu byla přičtena konstanta 1023 (bias).

Skutečný exponent maximálního čísla je tedy $2046 - 1023 = 1023$, exponent minimálního čísla je pak $1 - 1023 = -1022$.

Desítkové vyjádření

Desítkové vyjádření dvojkového čísla je přibližné s ohledem na omezený počet desítkových číslic.

Maximální mantisa je $\sum_{n=1}^{52} 2^{-n} \approx 0.99999999999999998889776975375$ na 28 platných číslic zlomku.

Zkreslený exponent je $\sum_{n=1}^{11} 2^n = 2046$, skutočný exponent je 1023.

$$\text{Maximum je } 2^{1023} \times (1 + (1 - 2^{52})) = 2^{1023} \times \sum_{n=0}^{52} 2^{-n} \approx 1.9999999999999998889776975375 \times 2^{1023} \approx 1.797693134862315508561243284\text{E}+308.$$

Minimum je $1 \times 2^{-1022} \approx 2.225073858507201383090232717\text{E}-308$.

Rozsah typu `float` v Pythonu

Rozsah čísel typu float v Pythonu je určen na 17 platných číslic, kdy je zaručena přesnost převodu z dvojkového vyjádření do desítkového a naopak.

```
print(sys.float_info.max) # 1.7976931348623157e+308
print(sys.float_info.min) # 2.2250738585072014e-308
```

Bajtová data

Bajtový řetězec (bytes)

Bajtový řetězec připomíná textový řetězec, ale na rozdíl od něj není posloupností Unicode znaků, ale je posloupností *bajtů* (bytes). Bajt (byte) je adresovací jednotka a v paměti zabírá 8 bitů. Jako číslo bez znaménka je v rozsahu $0 - 255_{(10)}$ nebo $00000000_{(2)} - 11111111_{(2)}$ nebo $00_{(16)} - ff_{(16)}$. Jako číslo se znaménkem má rozsah $-127_{(10)} - +127_{(10)}$.

Bajtový řetězec je objekt (instance) třídy `bytes`. Vytváří se voláním vestavěné funkce (konstruktoru třídy) `bytes()`.

```
bytes([source[, encoding="utf-8"[, errors="strict"]]])
```

source textový řetězec
 číslo typu `int` určující počet bajtů pro pole šestnáctkových čísel `0x00`
 posloupnost čísel typu `int` (0 až 255) – seznam, `n`-tice, `range`
encoding kódování např. `'utf-8'` (předvolené), `'ascii'`, `iso8859_2`, aj., viz [zde](#)
errors `'strict'` předvolený - při chybě vydá zprávu
 `'ignore'` nehlásí chybu
 `'replace'` nahradí chybný znak otazníkem
 a [další](#)

Bajtový řetězec je *neměnný*, tedy dovoluje stejné operace jako textový řetězec.

```
b = bytes('abcd', encoding='ascii') # řetězec s ASCII znaky
print(b) # b'abcd'
print(b.hex()) # 61626364
print(type(b)) # <class 'bytes'>
```

```
b = bytes(5) # pět nulových bajtů
print(b) # b'\x00\x00\x00\x00\x00'
print("len:", len(b)) # 5
```

```
b = bytes(range(5)) # posloupnost 0 až 4
print(b) # b'\x00\x01\x02\x03\x04'
print(b[2:4]) # b'\x02\x03'
print(b[2:3]) # b'\x02'
print(b[2]) # 2
```

```
b = bytes([28,65,46,66,46,67,46])
print(b) # b'\x1cA.B.C.'
print("len:", len(b)) # len: 7
```

```

b = bytes('č','utf-8') # znak Unicode 'č' = '\u010d' má číslo 269 čili hex 10d
print('č:', b) # č: b'\xc4\x8d'

b = bytes(source='č', encoding='ascii', errors='replace')
print('replace:', b) # replace: b'?' – náhrada otazníkem

b = bytes(source='č', encoding='ascii', errors='strict') # chyba
UnicodeEncodeError: 'ascii' codec can't encode character '\u010d' in position 0:
ordinal not in range(128)

# převod bytes do řetězce hexadecimálních číslic (bytes -> str)
string = b.hex(' ', 1) # vložení mezery mezi dvojice hexadecimálních číslic
print("hex:", string) # hex: 1c 41 2e 42 2e 43 2e

# zpětný převod z řetězce do bytes (str -> bytes)
print(bytes.fromhex(string)) # b'\x1cA.B.C.'

```

ASCII znaky a desítková čísla

	32	!	33	"	34	#	35	\$	36	%	37	&	38	'	39	(	40	
)	41	*	42	+	43	,	44	-	45	.	46	/	47	0	48	1	49	2 50
3	51	4	52	5	53	6	54	7	55	8	56	9	57	:	58	;	59	< 60
=	61	>	62	?	63	@	64	A	65	B	66	C	67	D	68	E	69	F 70
G	71	H	72	I	73	J	74	K	75	L	76	M	77	N	78	O	79	P 80
Q	81	R	82	S	83	T	84	U	85	V	86	W	87	X	88	Y	89	Z 90
[	91	\	92	]	93	^	94	_	95	`	96	a	97	b	98	c	99	d 100
e	101	f	102	g	103	h	104	i	105	j	106	k	107	l	108	m	109	n 110
o	111	p	112	q	113	r	114	s	115	t	116	u	117	v	118	w	119	x 120
y	121	z	122	{	123		124	}	125	~	126		127					

ASCII znaky a šestnáctková čísla

0x20	!	0x21	"	0x22	#	0x23	\$	0x24	%	0x25	&	0x26	'	0x27	(	0x28			
)	0x29	*	0x2a	+	0x2b	,	0x2c	-	0x2d	.	0x2e	/	0x2f	0	0x30	1	0x31	2	0x32
3	0x33	4	0x34	5	0x35	6	0x36	7	0x37	8	0x38	9	0x39	:	0x3a	;	0x3b	<	0x3c
=	0x3d	>	0x3e	?	0x3f	@	0x40	A	0x41	B	0x42	C	0x43	D	0x44	E	0x45	F	0x46
G	0x47	H	0x48	I	0x49	J	0x4a	K	0x4b	L	0x4c	M	0x4d	N	0x4e	O	0x4f	P	0x50
Q	0x51	R	0x52	S	0x53	T	0x54	U	0x55	V	0x56	W	0x57	X	0x58	Y	0x59	Z	0x5a
[	0x5b	\	0x5c	]	0x5d	^	0x5e	_	0x5f	`	0x60	a	0x61	b	0x62	c	0x63	d	0x64
e	0x65	f	0x66	g	0x67	h	0x68	i	0x69	j	0x6a	k	0x6b	l	0x6c	m	0x6d	n	0x6e
o	0x6f	p	0x70	q	0x71	r	0x72	s	0x73	t	0x74	u	0x75	v	0x76	w	0x77	x	0x78
y	0x79	z	0x7a	{	0x7b		0x7c	}	0x7d	~	0x7e		0x7f						

Literál bajtového řetězce

Literál bajtového řetězce se zapisuje stejně jako literál textového řetězce, ale s předřazeným písmenem **b** nebo **B**. Znak uvnitř řetězce smí být jen v kódu ASCII, který je podmnožinou Unicode, v pořadových číslech 0 až 127.

```
bajty = b"abcd"  
print(bajty)  # b'abcd'
```

Čísla 0 až 31 nepředstavují viditelné znaky, takže je nelze zapisovat z klávesnice. Můžeme je však zapsat jako dvojice šestnáctkových číslic s předponou `\x`. Tento zápis nejlépe odpovídá tomu, že jednotlivé prvky řetězce jsou spíše šestnáctková (hexadecimální) čísla než znaky.

```
bajty = b"\x01\x19\x15\x17"  
print(bajty)  # b'\x01\x19\x15\x17'
```

Oba způsoby zápisu lze kombinovat.

```
bajty = b"a\x62c\x64"  
print(bajty)  # b'abcd'
```

Kódování UTF-8

Formát UTF-8 ukládá znaky do jednoho až čtyř bajtů podle velikosti pořadového čísla Unicode. Následující tabulka ukazuje způsob kódování. Je-li nejvýznamnější bit bajtu 0, je znak uložen v jednom bajtu a odpovídá normě ASCII. V ostatních případech jsou rozhodující příznakové bity (markery) na začátku *prvního* bajtu: počet jedničkových bitů zakončených nulou (110, 1110, 11110) odpovídá počtu bajtů znaku (2, 3, 4), *ostatní* bajty začínají bity 10. Další bity v bajtu označené písmenem *x* slouží k vyjádření vlastního kódu znaku.

Čísla Unicode	bajt 1	bajt 2	bajt 3	bajt 4
U+0000 — U+007f	0xxxxxxx			
U+0080 — U+07ff	110xxxxx	10xxxxxx		
U+0800 — U+ffff	1110xxxx	10xxxxxx	10xxxxxx	
U+1000 — U+10ffff	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

Při transformaci **Unicode** → **UTF-8** se postupuje zprava doleva. Vyjdeme z pořadového čísla Unicode a jeho dvojkového vyjádření, například pro znak pejska. Podle hořejší tabulky bude mít kód UTF-8 čtyři bajty.

 U+1f436, 128054, So, DOG FACE
1 1111 0100 0011 0110 = 0x1f436 = '\u1f436'

Postupujeme zprava doleva. Rozčleníme bity po šesti a před šestice vsuneme bity 10, čímž se všechny bity roztáhnou doleva. Po doplnění nulami máme číslo ve 4 bajtech.

0000 0001 1111 0100 0011 0110 doplnění nulami do tří bajtů
000000|011111|010000|110110 skupiny po 6 bitech zprava
00000000**10**011111**10**010000**10**110110 bity 10 před šestice, číslo je teď ve 4 bajtech

Nakonec přepíšeme nejvyšší pozice bity 11110.

11110000**10**011111**10**010000**10**110110 bity 11110 do prvních 5 bitů prvního bajtu
1111 0000 1001 1111 1001 0000 1011 0110 rozčlenění po 4 bitech
f 0 9 f 9 0 b 6 = 0xf09f90b6 šestnáctkové číslo kódu UTF-8

Při transformaci **UTF-8** → **Unicode** odstraníme markery, čímž spojíme bity.

11110000**10**011111**10**010000**10**110110
000011111010000110110 = 1 1111 0100 0011 0110 = 0x1f436 pořadové číslo Unicode

Jiný příklad, písmeno č.

Unicode → UTF-8

č U+10d, 269, Ll, LATIN SMALL LETTER C WITH CARON
1 0000 1110 = 0x10d = '\u10d'

Do spodního bajtu čísla se na začátek vsunou bity 10 a zbyte v něm *šest* nejnižších bitů pořadového čísla. Zbývající vyšší bity čísla se posunou vlevo do horního bajtu.

00000001|00001110 = 0x010d
00000100|**10**001110 = 0x048d

Horní bajt se přepíše zleva bity 110.

`1100010010001110` = `0xc48d` šestnáctkové číslo kódu UTF-8

Stejný výsledek jsme získali výše použitím funkce `bytes()`.

```
b = bytes('č', 'utf-8') # znak Unicode 'č' = '\u010d' má číslo 269 čili hex 10d
print('č:', b) # č: b'\xc4\x8d'
```

UTF-8 → Unicode – odstraníme markery, čímž spojíme bity.

`1100010010001110`
`001000011110` = `1 0000 1110` = `0x10d` pořadové číslo Unicode

Podrobnosti viz <https://en.wikipedia.org/wiki/UTF-8>.

Znaky, které mají větší číslo, jsou kódovány dvojicí (někdy i jiným počtem) jiných znaků. Třeba česká vlajka  je kódována těmito dvěma znaky:

`U+1f1e8, 127464, So, REGIONAL INDICATOR SYMBOL LETTER C`
`Z U+1f1ff, 127487, So, REGIONAL INDICATOR SYMBOL LETTER Z`

a ve formátu UTF-8 zabírá 8 bajtů:

`0xf09f87a8f09f87bf`

Takové znaky v textu rozeznává Python *při čtení dopředu*, ale při čtení pozpátku je pochopitelně nemůže rozeznat.

```
st = '🇨🇪🇵🇱'
for c in range(len(st)):
    print(st[c], end='')
```

🇨🇪🇵🇱

```
for c in range(len(st)-1, 0, -1):
    print(st[c], end='')
```

🇪🇵🇨🇪

Texty s písmeny nebo jednoznakovými emotikony však rozeznává Python *i při čtení pozpátku*.

```
st = 'pozpátku: 🐛³λ%'
for c in range(len(st)):
    print(st[c], end='')
```

pozpátku: 🐛³λ%

```
for c in range(len(st)-1, -1, -1):
    print(st[c], end='')
```

%λ³🐛 :uktápzop

Bajtové pole (bytearray)

Bajtové pole je objekt třídy `bytearray`. Vytváří se voláním vestavěné funkce (konstruktoru třídy) `bytearray( )` s parametry. Nemá vlastní literál, ale může literál bajtového řetězce použít jako parametr.

```
bytearray([source[, encoding[, errors]]])
```

Od bajtového řetězce se liší jen tím, že je *měnitelné*, tedy že dovoluje stejné operace jako seznam (list). Do položek se mohou dosazovat jen celá čísla 0 až 255, třeba i ve formě `\x00` až `\xff`.

```
bar = bytearray(b'\x00\x01\x02\x03\x04')
print(bar)  # bytearray(b'\x00\x01\x02\x03\x04')

bar = bar[0:3]
print(bar)  # bytearray(b'\x00\x01\x02')

bar.append(0xff)
print(bar)  # bytearray(b'\x00\x01\x02\xff')

bar.append(0)
print(bar)  # bytearray(b'\x00\x01\x02\xff\x00')
```

Převod čísla typu `int` na bajtový řetězec a opačně

Převod celého čísla na bajty (instanční) metodou `to_bytes()` třídy `int`.

`to_bytes(length, byteorder, *, signed=False)`

length počet bajtů
byteorder povinný parametr = pořadí bajtů 'big' nebo 'little' nebo `sys.byteorder`
big - první bajt je nejvyšší (big endian)
little - první bajt je nejnižší (little endian)
sys.byteorder - systémové pořadí bajtů
signed False - číslo bez znaménka, True - číslo se znaménkem

Převod bajtů na celé číslo *třídní* metodou `int.from_bytes()` třídy `int` (class method).

`int.from_bytes(bytes, byteorder, *, signed=False)`

bytes objekt typu `bytes`, `bytearray`.
byteorder povinný parametr = pořadí bajtů 'big' nebo 'little' nebo `sys.byteorder`
big - první bajt je nejvyšší (big endian)
little - první bajt je nejnižší (little endian)
sys.byteorder - systémové pořadí bajtů
signed False - číslo bez znaménka, True - číslo se znaménkem

Poznámka 1: Pojmy instanční metoda a třídní metoda se probírají v další části kurzu o třídách.

```
# převod int na bajtový řetězec instanční metodou
i = -255
bajty = i.to_bytes(2, byteorder='little', signed=True)
print(bajty)    # b'\x01\xff'

# převod bajtového řetězce na int se znaménkem třídní metodou
i = int.from_bytes(bajty, byteorder='little', signed=True)
print(i)    # -255

bajty = i.to_bytes(2, byteorder='big', signed=True)
print(bajty)    # b'\xff\x01'

# převod bajtového řetězce na int bez znaménka třídní metodou
import sys
i = int.from_bytes(bajty, byteorder=sys.byteorder) # little
print(i)    # 511 (256 + 255)
```

Převod čísla typu **float** na bajtový řetězec a opačně

Při převodu zlomkového čísla typu `float` do bajtového řetězce použijeme modul **struct** a jeho dvě funkce `pack()` a `unpack()`. Ty používají jako první argument řetězec formátu. Formát je popsán v dokumentaci <https://docs.python.org/3/library/struct.html>.

pack(format, v1, v2, ...) vrací objekt typu `bytes`

format: "pořadí-bajtů[počet]formátový-znak..."
pořadí-bajtů = nebo @ – systémové (při vynechání platí @)
> big-endian
< little-endian
počet počet hodnot pro daný formátový znak
formátový-znak d 8 bajtů - C double
f 4 bajty - C float
l 4 bajty - C unsigned long
L 4 bajty - C unsigned long
q 8 bajtů - C unsigned long long
Q 8 bajtů - C unsigned long long

v1, v2, ... hodnoty čísel v počtu daném ve formátu

unpack(format, buffer) vrací **n-tici** hodnot typu určeného formátem

buffer: objekt typu `bytes` odpovídající formátu, jako by vznikl z funkce `pack()`.

```
from struct import *
```

Převod **float** na 8bajtový řetězec

```
bajty = pack('@2d', 1.45, 2.2) # C double – system byte order (little endian)
print(bajty) # b'3333333\xfb7?\x9a\x99\x99\x99\x99\x99\x01@'
```

Převod 8bajtového řetězce na **float**

```
d = unpack('2d', bajty) # system byte order (little endian)
print(d) # (1.45, 2.2) – n-tice
```

Převod **float** na 4bajtový řetězec

```
bajty = pack('<2f', 1.45, 2.2) # C float – little endian
print(bajty) # b'\x9a\x99\xb9?\xcd\xcc\x0c@'
```

Převod 4bajtového řetězce na **float**

```
f = unpack('<2f', bajty) # little endian
print(f) # (1.4500000476837158, 2.200000047683716) – n-tice
```

Převod float na 64 bitů textu

```
b = pack(">d", 1.45) # převod C double na bajty – big endian
(d,) = unpack(">Q", b) # převod bajtů na C long long – celé číslo 64bitové
print(f'{d:0{64}b}') # převod čísla na 64 znaků nul a jedniček (formát b)
```

```
0011111111110111001100110011001100110011001100110011001100110011
```

Červená číslice 0 představuje kladné znaménko (1 je záporné znaménko), šedě vyznačené číslice (11 bitů) představují modifikovaný exponent (zvětšený o 1023), ostatní číslice představují mantisu (52 bitů).

Převod float na 32 bitů textu

```
b = pack(">f", 1.45) # převod C float na bajty – big endian
f = unpack(">L", b)[0] # převod bajtů na C long – celé číslo 32bitové
print(f'{f:0{32}b}') # převod čísla na 32 znaků nul a jedniček (formát b)
```

```
001111111101110011001100110011010
```

Červená číslice 0 představuje kladné znaménko (1 je záporné znaménko), šedě vyznačené číslice (osm bitů) představují modifikovaný exponent (zvětšený o 127), ostatní číslice představují mantisu (23 bitů).

Převod 64 bitů textu na float

```
bin_string = '001111111111011100110011001100110011001100110011001100110011'
b = int(bin_string, 2).to_bytes(8, byteorder="big") # int -> bajty
d = unpack('>d', b)[0]
print(d) # 1.45
```

Převod 32 bitů textu na float

```
bin_string = '001111111101110011001100110011010'
b = int(bin_string, 2).to_bytes(4, byteorder="big") # int -> bajty
f = unpack('>f', b)[0]
print(f) # 1.4500000476837158
```

Bitové operace

Bitové operace se provádějí jen s *celými čísly* typu `int` na jejich jednotlivých bitech. Slouží k nim tyto operátory:

```
&    bitový součin AND
|    bitový součet OR
^    výlučný bitový součet XOR (exclusive OR)
~    zápor NOT (v Pythonu je to dvojkový doplněk)
<<  posun čísla vlevo
>>  posun čísla vpravo
```

Funkce `int(x, base=10)` má nepovinný druhý argument označující základ číselné soustavy, v níž je vyjádřen první argument, který pak musí být v řetězcové nebo bajtové podobě:

```
byte1 = int(b'11110000', base=2) # 240
byte2 = int(b'00001111', base=2) # 15
byte2 = int(b'0f', base=16)      # 15
```

nebo třeba

```
byte1 = int('11110000', 2) # 240
byte2 = int('00001111', 2) # 15
byte2 = int('0f', 16)      # 15
```

Bitový součin AND

```
print(f" {byte1:08b}") # šířka 8 znaků, typ b (binární formát)
print(f"& {byte2:08b}") # 0 – tiskne vedoucí nuly
print(f"= {byte1 & byte2:08b}") # & bitový součin
  11110000
& 00001111
  00000000
```

Bitový součet OR

```
print(f" {byte1:08b}")
print(f"| {byte2:08b}")
print(f" {byte1 | byte2:08b}") # | bitový součet
  11110000
| 00001111
  11111111
```

Výlučný bitový součet XOR

```
byte1 = int('11110000', 2) # 240
byte3 = int('01010101', 2) # 85
print(f" {byte1:08b}")
print(f"^ {byte3:08b}")
print(f" {byte1 ^ byte3:08b}") # ^ výlučný bitový součet
  11110000
^ 01010101
  10100101
```

Posuvy vpravo

```
byte2 = int('00001111', 2) # 15
print(f">> 1 {byte2 >> 1:08b}")
>> 1  00001111
      00000111
```

```
byte4 = int('11111111', 2) # 255
print(f">> 7 {byte4 >> 7:08b}")
>> 7  11111111
      00000001
```

Posuvy vlevo

```
byte2 = int('00001111', 2) # 15
print(f"<< 2 {byte2 << 2:08b}")
<< 2  00001111
      00111100  ( $15 \times 2^2 = 60$ )
```

```
byte4 = int('11111111', 2) # 255
print(f"<< 7 {byte4 << 7:08b}")
<< 7  11111111
1111111110000000  ( $255 \times 2^7 = 32640$ )
```

Zápor NOT

```
byte2 = int('00001111', 2) # 15
print(f"~ {~byte2:09b}") # tisknu 9 znaků
~ 00001111
-00010000  (-16)
```

Poznámka: Operace bitového záporu – negace – obrátí jedničky na nuly a nuly na jedničky. Protože však typ `int` má v nejvyšším bitu znaménko, vyjde v příkladu záporné číslo ve *dvojkovém doplňku*:

$\sim 00001111 \rightarrow 11110000 \rightarrow -(00001111 + 1) \rightarrow -00010000 = -16$

Definice bitového záporu (inverze) je podle dokumentace $\sim x = -(x + 1)$.

Testování bitu (AND)

```
maska = int('00000001', 2) # bit 1
# je zapnut bit 1?
print(f"{maska & int('11110000', 2):08b}") # 00000000 - False
print(f"{maska & int('00001111', 2):08b}") # 00000001 - True
```

Vložení bitu (OR)

```
print(f"{int('00000001', 2) | int('11110000', 2):08b}") # 11110001
```

Binární soubory

Na rozdíl od textových souborů, které se skládají ze znaků Unicode, je jejich obsahem *posloupnost bajtů*. Aplikace, která se souborem pracuje, musí znát strukturu jednotlivých bajtů nebo jejich skupin. Údaje o délce nebo počtu skupin bývají na začátku takových skupin.

Obsahem souboru může být třeba posloupnost čísel typu `int` (délky 1, 2, 4 nebo 8 bajtů) nebo čísel typu `float` o čtyřech nebo osmi bajtech apod. Vždy je třeba znát, kolik bajtů číslo zabírá a jakého je typu.

Soubor celých čísel

Vytvoření binárního souboru z celých čísel typu `int` (C short)

Při vícebajtových číslech je třeba dodržet pořadí bajtů (big nebo little endian).

```
cisla_int = (23, 32, 42, 20, 19, 6, 42)
with open('int.bin', 'wb') as outfile: # zápis čísel do souboru
    for i in cisla_int: # C short - 2 bajty
        b = i.to_bytes(2, byteorder='big', signed=True) # převod na 2 bajty
        outfile.write(b) # zápis 2 bajtů do souboru
        print(b.hex(), end=" ")
0017 0020 002a 0014 0013 0006 002a
```

Obsah nového souboru je 00170020002A001400130006002A (šestnáctkově).

Výpočet maxima, součtu a průměru z čísel typu `int` (C short)

```
import sys

with open('int.bin', 'rb') as infile:
    maximum = -sys.maxsize
    soucet = 0
    pocet = 0
    b = infile.read(2) # čtu první dva bajty
    while b != b'': # dokud není konec souboru
        i = int.from_bytes(bytes=b, byteorder='big', signed=True)
        if i > maximum:
            maximum = i
        soucet += i
        pocet += 1
        b = infile.read(2) # čtu další dva bajty
    print("\nmaximum =", maximum)
    print("soucet =", soucet)
    print("prumer =", soucet/pocet)

maximum = 42
soucet = 184
prumer = 26.285714285714285
```

Soubor čísel typu `float`

```
from struct import *
```

Vytvoření binárního souboru z čísel typu `float` (C double)

```
cisla_float = (3.1, 892.23e-2)
with open('float.bin', 'wb') as outfile:
    for double in cisla_float:
        b = pack('d', double) # little endian, 8bajtové číslo C double
        outfile.write(b) # zápis 8 bajtů do souboru
        print(f'{b.hex()}', end=" ")
cdcccccccccc0840 9c33a2b437d82140
```

Obsah nového souboru je `CDCCCCCCCCC08409C33A2B437D82140` (šestnáctkově).

Výpočet maxima z čísel typu `float` (C double)

```
with open('float.bin', 'rb') as infile:
    maximum = float('-inf')
    b = infile.read(8) # čtu prvních 8 bajtů
    while b != b'': # dokud není konec souboru
        d = unpack('d', b)[0] # little endian, nultý člen jednočlenné n-tice
        if d > maximum:
            maximum = d
        print(f'{b}={d}', end=" ")
        b = infile.read(8) # čtu dalších 8 bajtů
    print("\nmaximum =", maximum)
```

```
b'\xcd\xcc\xcc\xcc\xcc\xcc\x08@'=3.1 b'\x9c3\xa2\xb47\xd8!@'=8.9223
maximum = 8.9223
```